

Performance Testing

Date	02 July 2026
Team ID	xxxxxx
Project Name	Voice-Based Concept Understanding Analyser
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman / Locust
Type of Testing	Load Testing & Stress Testing
Target Module / API	Audio Upload and AI Evaluation Endpoint (/evaluate)
Test Environment	Local Server (Uvicorn / FastAPI)
Test Date	02 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Light Load: Single student uploading a 1-minute audio file.	1	10	Response within 2 seconds with accurate text and score.
2	Normal Academic Load: Multiple students submitting answers simultaneously at the end of a class.	20	60	System processes all requests without dropping connections.
3	Peak Load / Stress Test: Simulating a whole lab batch uploading files at the exact same time.	50	120	System queues requests and handles them gracefully without crashing.
4	Database History Stress: Multiple users concurrently fetching past evaluation history logs.	30	30	SQLite database returns records instantly without locking.

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.5 seconds	Pass	Fast response for transcription and core fluency metrics.
2	Response Time (Max)	< 5 seconds	4.2 seconds	Pass	Max time taken only when waiting for Gemini API response.
3	Throughput (Req/sec)		0%	Pass	All audio uploads and API calls processed successfully.
4	Error Rate	< 1%	45%	Pass	Lightweight FastAPI backend keeps CPU consumption low.
5	CPU Utilization	< 80%	15 req/sec	Pass	Handles multiple concurrent audio submissions effectively.
6	Memory Utilization	< 80%	60%	Pass	Temporary audio files are cleared from memory post-processing.

Step 4: Observations & Analysis

Key Findings:
 [Describe what the test results revealed about your system's performance]

Bottlenecks Identified:
 [List any performance bottlenecks observed during testing]

Optimization Steps Taken:
 [Describe any changes or optimizations made to improve performance]

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

Code

Details

200

Response body

```
{
  "student_name": "Sneha",
  "filename": "ai_voice.mp4",
  "transcription": "a is a branch of computer science focused on building software and computer systems capable of performing tasks that traditionally require human intelligence these tasks include learning from experience recognising complex patterns understanding language and making autonomous decisions instead of following rigid pre return code AI models process massive datasets using advanced algorithms to find insights and adopt over time the course of fields of air machine learning deep learning natural language processing and computer vision machine learning is the foundational process where computers analysed data to find patterns and improve a tasks without direct program",
  "understanding_score": "92%",
  "feedback": "Strong Understanding",
  "filler_stats": {
    "um": 0,
    "uh": 0,
    "like": 0,
    "ah": 0
  },
  "audio_metrics": {
    "duration_sec": 64.3,
    "word_count": 96,
    "words_per_minute": 89,
    "pause_ratio": "17.58%",
    "similarity_score": "90.72%"
  },
  "database_status": "Saved to Database successfully!"
}
```

Download

Response headers

```
access-control-allow-credentials: true
access-control-allow-origin: http://127.0.0.1:8000
content-length: 1026
content-type: application/json
date: Sat, 11 Jul 2026 09:49:24 GMT
server: uvicorn
vary: Origin
```

```
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:      Started reloader process [10008] using StatReload
INFO:      Started server process [4924]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO: 127.0.0.1:10759 - "GET /docs HTTP/1.1" 200 OK
INFO: 127.0.0.1:10759 - "GET /openapi.json HTTP/1.1" 200 OK
INFO: 127.0.0.1:11838 - "POST /upload-audio/ HTTP/1.1" 200 OK
```